

Multi-Agent Systems: Homework Assignment 2

Alexandru Licuriceanu
alicuriceanu@stud.acs.upb.ro

1 EnvironmentManager

- **get_rooms**

- Constructs the target URL for the home workspace and issues an HTTP GET request to the simulator.
- Parses the resulting response text into an RDF graph using rdflib with the turtle format.
- Queries the graph for triples matching the hmas:contains predicate to identify child entities within the home.
- Filters the results for URIs containing the #workspace fragment, extracts the room name from the URI path, and returns the list of rooms.

- **get_artifacts_in_room**

- Targets a specific room by appending the room name to the home workspace URL and fetching its RDF description via HTTP GET.
- Parses the fetched Turtle document into an RDF graph.
- Iterates through hmas:contains relationships and filters objects that include the #artifact fragment to isolate smart devices.
- Returns a list of the full artifact URIs located in that specific room.

- **get_artifact_affordances**

- Fetches the Web of Things (WoT) Thing Description (TD) of the target artifact using an HTTP GET request.
- Identifies the abstract device type (e.g. Fan, Light) by checking rdf:type triples against the http://example.org/ namespace, explicitly filtering out room classifications.
- Parses Property Affordances by finding td:hasPropertyAffordance nodes, extracting their human-readable td:name and the hctl:hasTarget URI from their form descriptions.
- Parses Action Affordances via td:hasActionAffordance, extracting target URIs and mapping parameter requirements by iterating through td:hasInputSchema and jsonschema:properties.
- Packages and returns this data as an ArtifactInfo object.

- **get_artifact_state**
 - First calls `get_artifact_affordances` to retrieve the list of readable properties for the device.
 - Iterates through the properties, optionally filtering for a specific `property_name` if provided in the arguments.
 - Invokes the `read_property` helper method to send an HTTP GET request to each property’s target URI.
 - Unwraps the JSON response (extracting the “value” key if it exists, or storing errors/raw values) and populates an `ArtifactState` object to return.
- **get_active_preferences**
 - Iterates through the internal list of user preference constraints loaded during the agent’s initialization.
 - Extracts the start and end times from each preference’s `dislike_interval`.
 - Utilizes native Python string comparison (e.g. `start_time <= issued_at < end_time`) to determine if the constraint is active, taking advantage of the zero-padded “HH:MM” format.
 - Appends matching preferences to a list and returns the currently active subset.

2 FullContextSolver

The `FullContextSolver` implements a “Full Context” strategy, relying on a single, comprehensive LLM prompt to process smart home requests. Rather than selectively probing the environment, this solver acts as a complete knowledge gatherer. It queries the simulator for every room, device, state, and active preference in the entire house, aggregates this data into a single JSON context, and delegates the entire reasoning process to one monolithic LLM call. Below are the implementation details:

- The solver begins by querying the `EnvironmentManager` for every available room in the target home using `get_rooms()`.
- It iterates through every discovered room to fetch all contained artifacts. For each artifact, it retrieves both the available action schemas (`get_artifact_affordances()`) and the current property values (`get_artifact_state()`). This is compiled into a nested `environment_state` dictionary representing the entire house.
- Active constraints are fetched using `get_active_preferences(request.issued_at)` and formatted into a concise list of blocked device-room pairings.
- The solver constructs a large prompt that feeds the LLM the user request, timestamp, active constraints, and the complete `environment_state` JSON.
- The prompt enforces rigid constraints on the LLM, demanding strict device matching, exact parameter mapping based on the provided schemas, and stateless math (treating percentage adjustments as absolute numerical changes applied to the initial state).
- The solver expects a strict JSON array from the LLM. It utilizes regular expressions to strip away markdown formatting, extracts the JSON, and maps each sub-goal into an `ActionOutput` object (either success with an affordance URI and parameters, or `error_input` if an action is invalid or blocked).

3 SequentialSolver

The SequentialSolver implements a “Sequential Exploration” strategy. Rather than flooding the LLM with the entire state of the smart home, it utilizes a two-step reasoning approach to minimize context size and reduce token usage. It first infers the relevant target rooms from the natural language request, and then selectively builds a state representation exclusively for those specific rooms before generating the final action commands.

- The solver first retrieves the list of all available rooms in the home and sends a targeted prompt to the LLM. The LLM is tasked with extracting only the specific rooms mentioned or implied in the user’s request.
- Instead of querying the entire house, the agent iterates strictly over the extracted valid rooms. For each relevant room, it queries the environment to discover artifacts, their specific affordances, and their current property states.
- Active constraints are fetched based on the request’s timestamp to ensure no blocked actions are considered by the LLM.
- The solver constructs a highly focused second prompt. This prompt feeds the LLM the user request, the active constraints, and the heavily filtered environment state containing only the devices in the requested rooms.
- The action-mapping prompt enforces rigid constraints, demanding strict device and parameter matching based on the provided schemas, and requiring stateless mathematical adjustments (e.g. treating percentages as absolute numerical changes against the initial state).
- The final LLM output is processed using regular expressions to safely strip away markdown formatting and extract the JSON array. The parsed sub-goals are then mapped into standard ActionOutput objects, with automatic fallbacks to `error_input` for any parsing failures or invalid executions.

4 SemanticSolver

The SemanticSolver implements a “Semantic Classification” strategy. Instead of forcing the LLM to process the entire environment state and generate exact simulator commands, it uses the LLM strictly as a natural language parser. The LLM translates the user’s request into a structured JSON array of semantic sub-goals, while deterministic Python logic handles the actual mapping of these goals to the simulator’s specific artifacts and affordances.

- The solver pre-fetches a summary of rooms and their contained device types to provide spatial context. It prompts the LLM with rigid vocabulary rules to classify intents as either absolute assignments (`set_property`) or relative mathematical changes (`adjust_property`), enforcing specific naming conventions (e.g. using “degree” for curtains).
- To prevent LLM hallucinations, the prompt instructs the model to leave the room blank if it is omitted in the request. The Python code then safely infers the location deterministically by scanning the environment for the first room containing the requested device type.

- Before proceeding with any sub-goal, the solver evaluates active constraints. It performs a strict equality check between the requested device and the preference constraints to reliably block unauthorized actions.
- For relative adjustments, the agent fetches the target device’s current state. It implements fuzzy-matching fallbacks (safely resolving the semantic “degree” to the simulator’s internal “position” state) to handle naming quirks, and defaults uninitialized states to zero before applying mathematical deltas.
- The solver maps the semantic intent to executable endpoints using a structured fallback hierarchy. It first attempts an exact schema match, falls back to fuzzy parameter matching, and finally employs endpoint routing to translate string-based states (“on”, “off”, “open”, “close”) directly into specific action URIs.

5 Results

After implementing and running all 3 solver methods, we aggregated all results in Table 1, shown below:

Table 1: Comparison of Solver Strategies

Metric	Full Context	Sequential	Semantic
Success Rate	83.33%	86.11%	94.44%
Quantifiable Rate	11.11%	8.33%	2.78%
Success + Quantifiable	94.44%	94.44%	97.22%
Action Precision	94.44%	97.06%	100.00%
Action Recall	89.47%	86.84%	97.37%
Action F1 Score	91.89%	91.67%	98.67%
Property Accuracy	100.00%	93.94%	100.00%
Impossible Detection Rate	93.33%	100.00%	100.00%

Note: Test Case 20

During a manual inspection of the evaluation logs, a specific discrepancy was identified in the ground truth dataset for Test Case 20 (home18_one_760).

The input request for this test case is: *“Turn off the dehumidifier in the guest bedroom.”*

The evaluation dataset incorrectly expects an error_input execution status for this command. However, querying the environment reveals that the artifact guestBedroomDehumidifiers is indeed instantiated within the guest bedroom and explicitly exposes a turn_off action affordance. In addition to this, time-based preference checks confirm there are no active constraints blocking the use of this device at the requested time (17:00).

Because the target device exists, the affordance is valid, and no preferences are violated, the solver correctly maps the action to the turn_off endpoint. The expectation of an error_input in the ground truth doesn’t really make sense and appears to be a flaw in the dataset itself rather than a failure of the solver’s reasoning logic.

6 Analysis & Conclusion

Based on the evaluation metrics (Table 1), the Semantic Classification Strategy is definitively the best-performing approach. It achieved the highest overall Success Rate (94.44%), a flawless Action Precision (100.00%), and a perfect Property Accuracy (100.00%).

In contrast, the Full Context Strategy struggled the most, achieving only an 83.33% Success Rate and an Impossible Detection Rate of 93.33%. The Sequential Exploration Strategy offered a slight improvement in overall success (86.11%) and successfully identified all impossible actions (100.00%) by filtering out irrelevant room data, but it still fell behind the Semantic approach in Action F1 Score and Property Accuracy.

- In the Full Context and Sequential approaches, the LLM is burdened with both natural language understanding and deterministic API routing. Forcing the model to read complex JSON schemas, calculate state changes, and output precise execution URIs leads to hallucinations, such as inventing property names or failing math operations on uninitialized states.
- The Semantic strategy succeeds by restricting the LLM to what it does best: semantic parsing. It is only responsible for translating human intent into a generic, structured format.
- Logic-based tasks such as mapping vocabulary to the correct schema, calculating exact numeric deltas, verifying preference constraints, and handling endpoint fallbacks are fully offloaded to native Python code.
- By removing the API schemas from the LLM’s context window entirely, execution hallucinations are successfully eliminated.

Lessons Learned

Developing these three solvers highlighted several critical lessons regarding the integration of LLMs into multi-agent environments:

- Large Language Models excel at extracting intent, entities, and parameters from unstructured text. However, they are highly unreliable when forced to strictly adhere to nested JSON schemas or perform mathematical adjustments. Delegating execution logic to deterministic code is significantly safer and more accurate.
- The Full Context strategy proved that providing an LLM with more data does not equate to better reasoning. Flooding the prompt with the entire state of a smart home introduced noise, causing the model to confuse devices across different rooms or misinterpret active preferences.
- A major hurdle discovered during testing was the LLM’s tendency to auto-correct infeasible user requests. If a user asked for a device that did not exist, the LLM would alter the request to match an existing device to avoid outputting an error. In agent applications, prompts must be strict, commanding the LLM to act as an objective parser that reports exactly what was said, allowing the system to handle failures and constraints gracefully.
- Real-world APIs and simulators are rarely semantically perfect. A user might say “degree”, but the simulator stores the state as “position”. A user might say “close”, but the API requires an integer of 0. Robust agent architectures require fuzzy-matching and alias dictionaries in the integration layer to bridge the gap between human language and machine APIs.